

Universidad Andrés Bello

Ingeniería en Automatización y Robótica

Trabajo Colaborativo N°2

Manejo de Entradas y Salidas Digitales

Asignatura: Microcontroladores

Código: AAUT103

Integrantes

Alex Díaz

María José Córdova

Índice

1. Introducción	1
2. Actividad 1: LED parpadeante	1
2.1. Programa utilizado	1
2.2. Resultado de la simulación	2
3. Actividad 2: Control de LED mediante pulsador	4
3.1. Programa realizado	4
3.2. Resultado de la simulación	5
4. Actividad 3: Control alternado de dos LED	7
4.1. Programa realizado	7
4.2. Resultado de la simulación	9
5. Actividad 4: Control de un LED mediante dos pulsadores	10
5.1. Programa realizado	10
5.2. Resultado de la simulación	12
6. Actividad 5: Semáforo con un PIC	13
6.1. Programa realizado	13
6.2. Fase 1	16
6.3. Fase 2	16
6.4. Fase 3	17
6.5. Fase 4	17
7. Dificultades y aprendizajes durante el desarrollo	19
8. Conclusión	20

1. Introducción

En este trabajo colaborativo se desarrollaron distintas actividades de programación y simulación utilizando el microcontrolador PIC16F877A. Para esto se utilizó MPLAB X junto al compilador XC8 para generar los archivos `.hex`, y SimulIDE para comprobar el funcionamiento de los circuitos.

Las actividades permitieron trabajar progresivamente con salidas digitales, entradas mediante pulsadores y el control de varios LED, hasta llegar a la implementación de un sistema de dos semáforos controlados mediante el microcontrolador.

Durante el desarrollo se trabajó con los registros de configuración de entradas y salidas, principalmente TRIS y PORT, además de la configuración necesaria para utilizar pines del puerto A como entradas digitales.

2. Actividad 1: LED parpadeante

En esta primera actividad se compiló y simuló un programa básico para controlar un LED conectado al pin RB0 del microcontrolador PIC16F877A.

El programa enciende el LED durante 500 milisegundos y posteriormente lo apaga durante otros 500 milisegundos, repitiendo este comportamiento de forma continua.

El código utilizado corresponde al programa entregado en la guía del trabajo colaborativo.

2.1 Programa utilizado

```
1  /*
2   * descripcion: programa basico para encender un led conectado
3   * al pin rb0 del pic16f877a
4   */
5
6  // configuracion del pic
7  #pragma config FOSC = XT
8  #pragma config WDTE = OFF
9  #pragma config PWRTE = ON
10 #pragma config BOREN = ON
11 #pragma config LVP = OFF
12 #pragma config CPD = OFF
13 #pragma config WRT = OFF
14 #pragma config CP = OFF
15
```

```
16 // libreria principal para usar el pic con xc8
17 #include <xc.h>
18
19 // frecuencia del cristal de 4 mhz
20 #define _XTAL_FREQ 4000000
21
22 // nombre que usaremos para el pin rb0
23 #define LED RB0
24
25 void main(void)
26 {
27     // configura todo el puerto b como salida
28     TRISB = 0x00;
29
30     // deja todas las salidas del puerto b apagadas al iniciar
31     PORTB = 0x00;
32
33     // el programa se repite para siempre
34     while (1)
35     {
36         // enciende el led
37         LED = 1;
38
39         // espera 500 milisegundos
40         __delay_ms(500);
41
42         // apaga el led
43         LED = 0;
44
45         // espera otros 500 milisegundos
46         __delay_ms(500);
47     }
48
49     return;
50 }
```

2.2 Resultado de la simulación

En la siguiente captura se puede observar la compilación correcta del programa en MPLAB X y la ejecución del circuito en SimulIDE.

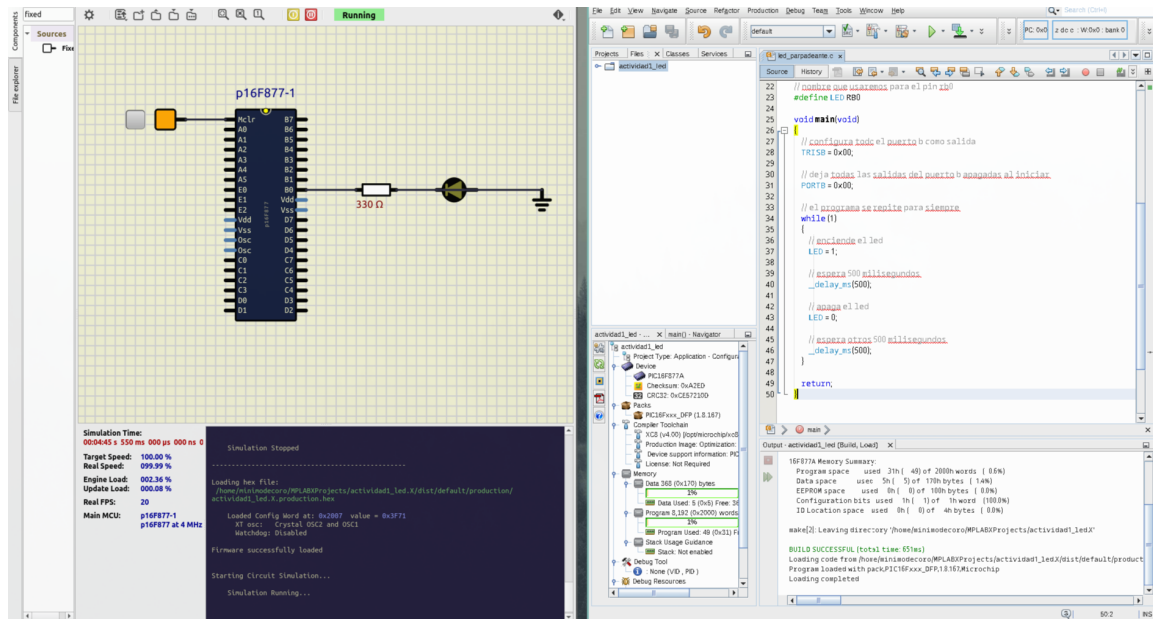


Figura 1: Compilación en MPLAB X y simulación del LED parpadeante en SimulIDE.

3. Actividad 2: Control de LED mediante pulsador

En esta actividad se utilizó un pulsador conectado al pin RA0 como entrada digital y un LED conectado al pin RB0 como salida.

El pulsador fue configurado mediante una resistencia pull-up, por lo que al encontrarse sin presionar la entrada permanece en estado lógico alto, mientras que al presionarlo la entrada cambia a estado lógico bajo.

El programa detecta este cambio y enciende el LED mientras el pulsador se encuentra presionado.

3.1 Programa realizado

```
1  /*
2   * descripcion: enciende un led cuando se presiona un pulsador
3   * y lo apaga cuando se suelta
4   */
5
6  // configuracion del pic
7  #pragma config FOSC = XT
8  #pragma config WDTE = OFF
9  #pragma config PWRTE = ON
10 #pragma config BOREN = ON
11 #pragma config LVP = OFF
12 #pragma config CPD = OFF
13 #pragma config WRT = OFF
14 #pragma config CP = OFF
15
16 // libreria principal para usar el pic con xc8
17 #include <xc.h>
18
19 // frecuencia del cristal
20 #define _XTAL_FREQ 4000000
21
22 // led conectado a rb0
23 #define LED RB0
24
25 // pulsador conectado a ra0
26 #define PULSADOR RAO
27
28 void main(void)
29 {
```

```
30 // convierte los pines analogicos del puerto a en digitales
31 ADCON1 = 0x06;
32
33 // rb0 sera salida para el led
34 TRISBbits.TRISB0 = 0;
35
36 // ra0 sera entrada para el pulsador
37 TRISAbits.TRISAO = 1;
38
39 // parte con el led apagado
40 PORTB = 0x00;
41
42 // el programa se repite para siempre
43 while(1)
44 {
45     // con pull-up, al presionar el boton ra0 pasa a 0
46     if(PULSADOR == 0)
47     {
48         // boton presionado
49         LED = 1;
50     }
51     else
52     {
53         // boton suelto
54         LED = 0;
55     }
56 }
57
58 return;
59 }
```

3.2 Resultado de la simulación

La primera captura muestra el LED apagado cuando el pulsador se encuentra sin presionar.

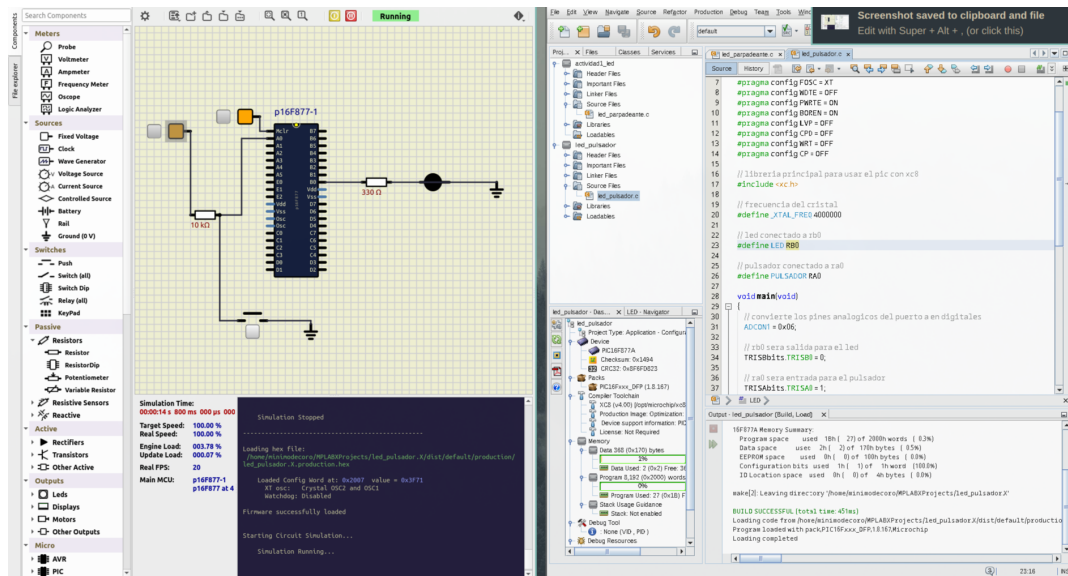


Figura 2: Actividad 2 con el LED apagado.

La segunda captura muestra el LED encendido al accionar el pulsador.

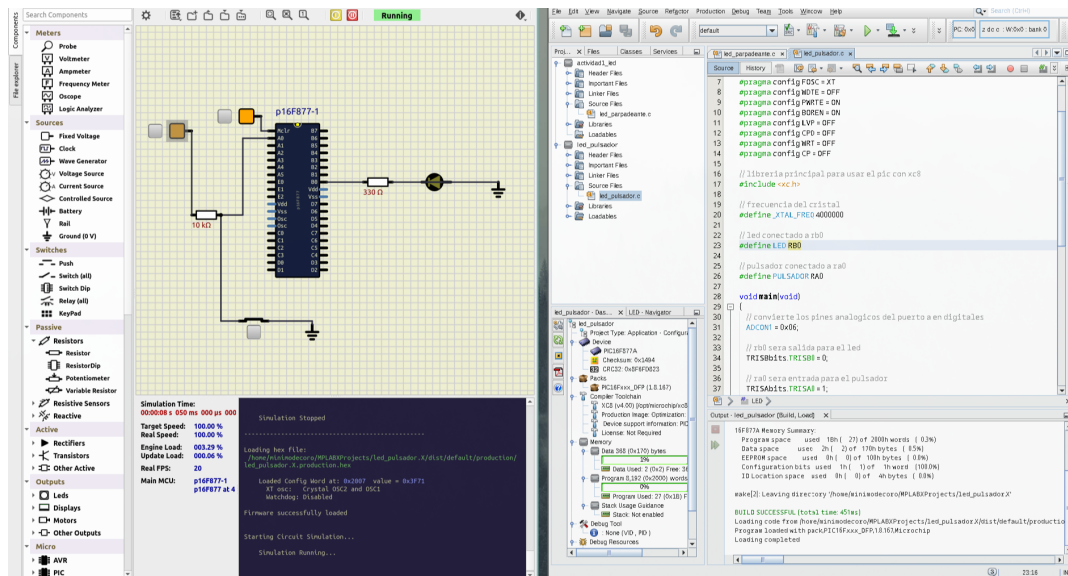


Figura 3: Actividad 2 con el LED encendido.

4. Actividad 3: Control alternado de dos LED

En esta actividad se utilizaron dos LED conectados a los pines RB0 y RB1.

El pulsador conectado a RA0 permite cambiar entre dos estados. En uno de ellos se encuentra encendido el primer LED y apagado el segundo, mientras que al presionar el pulsador estos estados se invierten.

4.1 Programa realizado

```
1  /*
2   * descripcion: controla dos leds de forma alternada
3   * mediante un pulsador
4   */
5
6  // configuracion del pic
7  #pragma config FOSC = XT
8  #pragma config WDTE = OFF
9  #pragma config PWRTE = ON
10 #pragma config BOREN = ON
11 #pragma config LVP = OFF
12 #pragma config CPD = OFF
13 #pragma config WRT = OFF
14 #pragma config CP = OFF
15
16 // libreria principal para usar el pic con xc8
17 #include <xc.h>
18
19 // frecuencia del cristal
20 #define _XTAL_FREQ 4000000
21
22 // leds conectados al puerto b
23 #define LED1 RB0
24 #define LED2 RB1
25
26 // pulsador conectado a ra0
27 #define PULSADOR RA0
28
29 void main(void)
30 {
31     // convierte los pines analogicos del puerto a en digitales
32     ADCON1 = 0x06;
```

```
33
34 // rb0 sera salida para el led 1
35 TRISBbits.TRISB0 = 0;
36
37 // rb1 sera salida para el led 2
38 TRISBbits.TRISB1 = 0;
39
40 // ra0 sera entrada para el pulsador
41 TRISAbits.TRISA0 = 1;
42
43 // deja el puerto b apagado al iniciar
44 PORTB = 0x00;
45
46 // el programa se repite para siempre
47 while(1)
48 {
49     // si el boton esta presionado
50     if(PULSADOR == 0)
51     {
52         // apaga el led 1
53         LED1 = 0;
54
55         // enciende el led 2
56         LED2 = 1;
57     }
58     else
59     {
60         // enciende el led 1
61         LED1 = 1;
62
63         // apaga el led 2
64         LED2 = 0;
65     }
66 }
67
68 return;
69 }
```

4.2 Resultado de la simulación

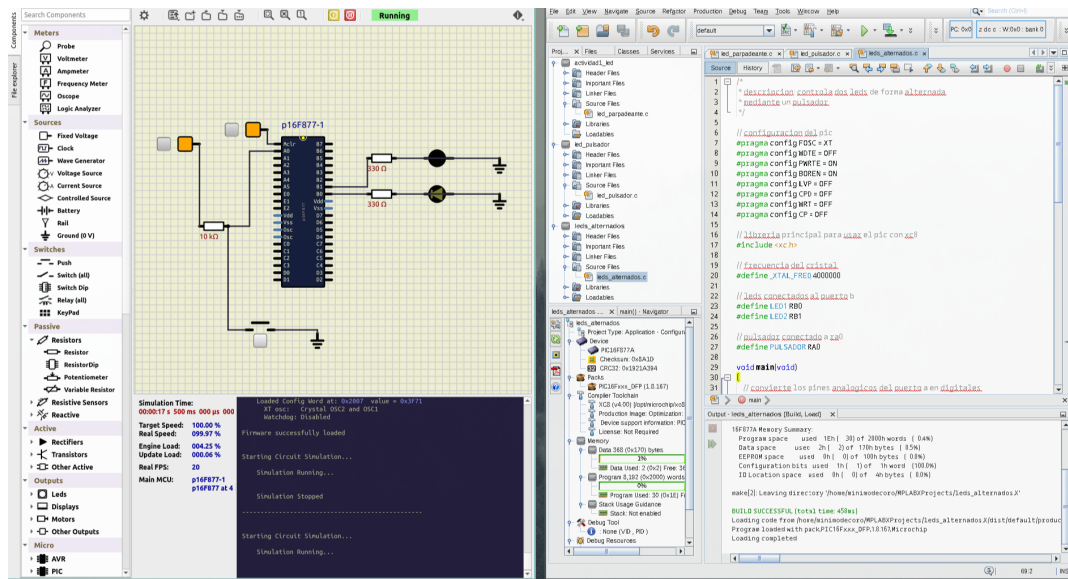


Figura 4: Primer estado de la actividad 3: LED 1 encendido y LED 2 apagado.

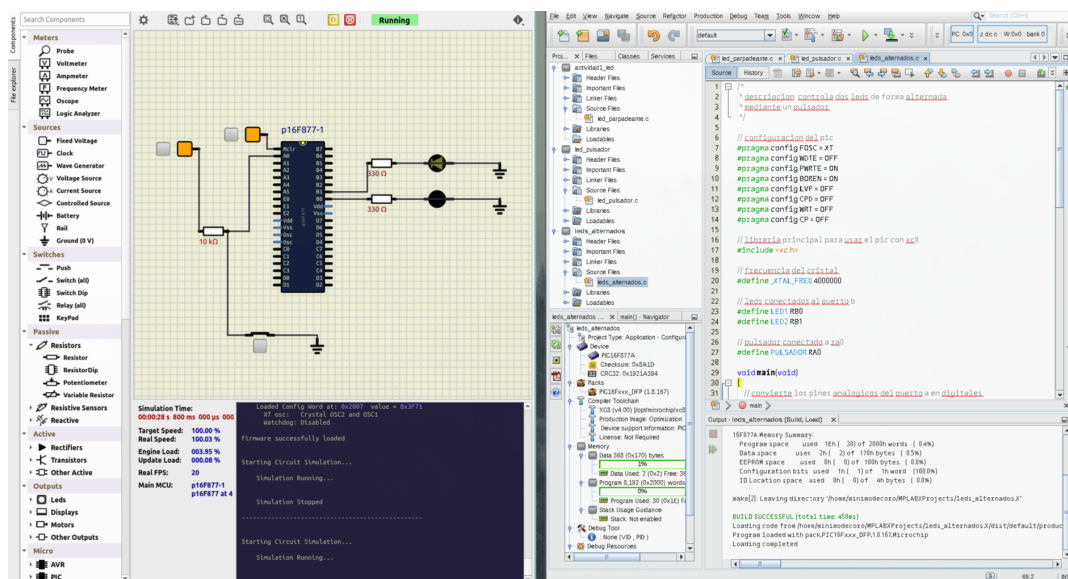


Figura 5: Segundo estado de la actividad 3: LED 1 apagado y LED 2 encendido.

5. Actividad 4: Control de un LED mediante dos pulsadores

Para esta actividad se utilizaron dos pulsadores independientes.

El primer pulsador se encuentra conectado al pin RA0 y permite encender el LED. El segundo se encuentra conectado al pin RA1 y permite apagarlo.

Una vez encendido, el LED mantiene su estado hasta que se acciona el pulsador de apagado.

5.1 Programa realizado

```
1  /*
2   * descripcion: enciende un led con un pulsador
3   * y lo apaga con otro pulsador
4   */
5
6  // configuracion del pic
7  #pragma config FOSC = XT
8  #pragma config WDTE = OFF
9  #pragma config PWRTE = ON
10 #pragma config BOREN = ON
11 #pragma config LVP = OFF
12 #pragma config CPD = OFF
13 #pragma config WRT = OFF
14 #pragma config CP = OFF
15
16 // libreria principal para usar el pic con xc8
17 #include <xc.h>
18
19 // frecuencia del cristal
20 #define _XTAL_FREQ 4000000
21
22 // led conectado a rb0
23 #define LED RB0
24
25 // pulsador de encendido conectado a ra0
26 #define BOTON_ON RAO
27
28 // pulsador de apagado conectado a ra1
29 #define BOTON_OFF RA1
```

```
30
31 void main(void)
32 {
33     // convierte los pines analogicos del puerto a en digitales
34     ADCON1 = 0x06;
35
36     // rb0 sera salida para el led
37     TRISBbits.TRISBO = 0;
38
39     // ra0 sera entrada para el boton de encendido
40     TRISAbits.TRISAO = 1;
41
42     // ra1 sera entrada para el boton de apagado
43     TRISAbits.TRISA1 = 1;
44
45     // el led parte apagado
46     LED = 0;
47
48     // el programa se repite para siempre
49     while(1)
50     {
51         // si se presiona el boton de encendido
52         if(BOTON_ON == 0)
53         {
54             LED = 1;
55         }
56
57         // si se presiona el boton de apagado
58         if(BOTON_OFF == 0)
59         {
60             LED = 0;
61         }
62     }
63
64     return;
65 }
```

5.2 Resultado de la simulación

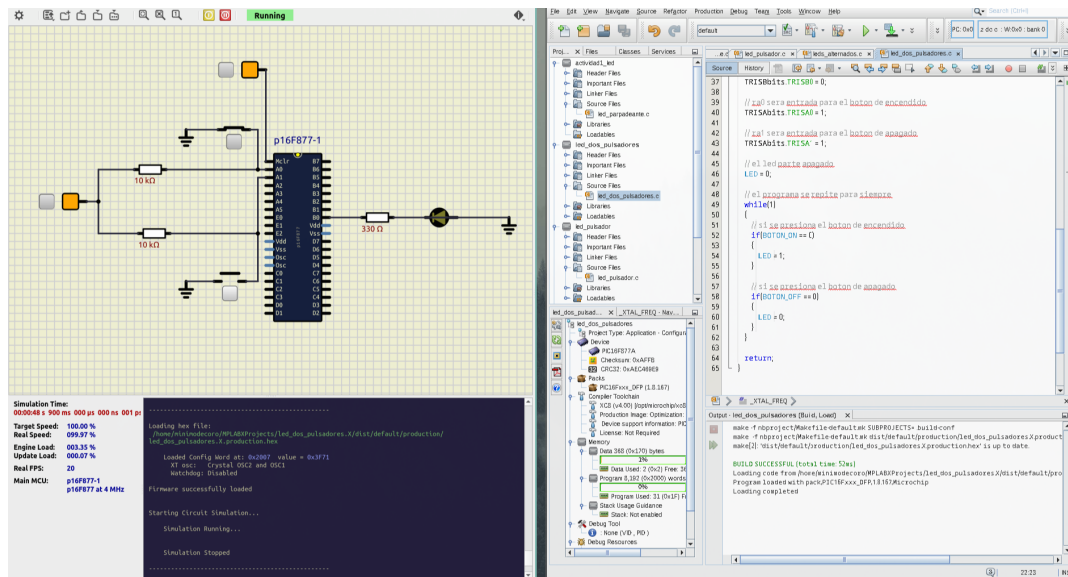


Figura 6: LED encendido mediante el pulsador conectado a RA0.

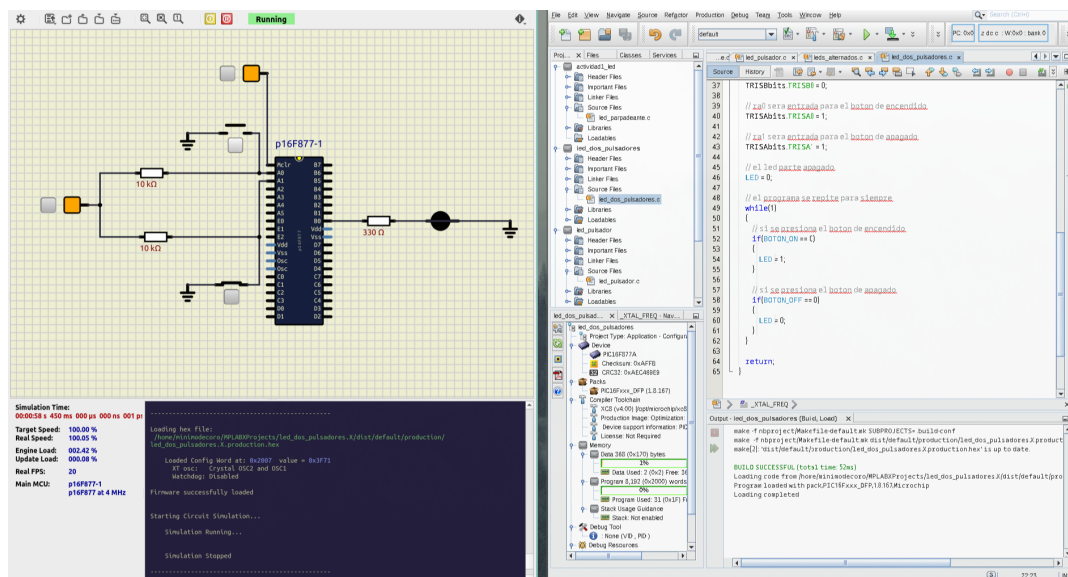


Figura 7: LED apagado mediante el pulsador conectado a RA1.

6. Actividad 5: Semáforo con un PIC

En la última actividad se implementó el control de dos semáforos utilizando seis LED.

El primer semáforo utiliza los pines RB0, RB1 y RB2 para sus luces verde, amarilla y roja respectivamente.

El segundo semáforo utiliza los pines RB3, RB4 y RB5.

Además, se utiliza una entrada en RA0 para activar la secuencia mediante un interruptor.

El programa ejecuta cuatro fases de un segundo cada una y repite el ciclo mientras el interruptor se encuentra activado.

6.1 Programa realizado

```
1  /*
2   * descripcion: controla dos semaforos con seis leds
3   * la secuencia funciona mientras el interruptor esta activado
4   */
5
6  // configuracion del pic
7  #pragma config FOSC = XT
8  #pragma config WDTE = OFF
9  #pragma config PWRTE = ON
10 #pragma config BOREN = ON
11 #pragma config LVP = OFF
12 #pragma config CPD = OFF
13 #pragma config WRT = OFF
14 #pragma config CP = OFF
15
16 // libreria principal para usar el pic con xc8
17 #include <xc.h>
18
19 // frecuencia del cristal
20 #define _XTAL_FREQ 4000000
21
22 // semaforo 1
23 #define VERDE1 RB0
24 #define AMARILLO1 RB1
25 #define ROJO1 RB2
26
27 // semaforo 2
28 #define VERDE2 RB3
```

```
29 #define AMARILLO2 RB4
30 #define ROJO2 RB5
31
32 // interruptor conectado a ra0
33 #define INTERRUPTOR RAO
34
35 void main(void)
36 {
37     // convierte los pines analogicos del puerto a en digitales
38     ADCON1 = 0x06;
39
40     // rb0 a rb5 seran salidas
41     TRISBbits.TRISB0 = 0;
42     TRISBbits.TRISB1 = 0;
43     TRISBbits.TRISB2 = 0;
44     TRISBbits.TRISB3 = 0;
45     TRISBbits.TRISB4 = 0;
46     TRISBbits.TRISB5 = 0;
47
48     // ra0 sera entrada para el interruptor
49     TRISAbits.TRISAO = 1;
50
51     // todos los leds parten apagados
52     PORTB = 0x00;
53
54     while(1)
55     {
56         // si el interruptor esta activado
57         if(INTERRUPTOR == 0)
58         {
59             // fase 1
60             // semaforo 1 verde
61             // semaforo 2 rojo
62
63             VERDE1 = 1;
64             AMARILLO1 = 0;
65             ROJO1 = 0;
66
67             VERDE2 = 0;
68             AMARILLO2 = 0;
69             ROJO2 = 1;
70
71             __delay_ms(1000);
```

```
72
73     // fase 2
74     // semaforo 1 amarillo
75     // semaforo 2 rojo
76
77     VERDE1 = 0;
78     AMARILLO1 = 1;
79     ROJO1 = 0;
80
81     VERDE2 = 0;
82     AMARILLO2 = 0;
83     ROJO2 = 1;
84
85     __delay_ms(1000);
86
87     // fase 3
88     // semaforo 1 rojo
89     // semaforo 2 verde
90
91     VERDE1 = 0;
92     AMARILLO1 = 0;
93     ROJO1 = 1;
94
95     VERDE2 = 1;
96     AMARILLO2 = 0;
97     ROJO2 = 0;
98
99     __delay_ms(1000);
100
101     // fase 4
102     // semaforo 1 rojo
103     // semaforo 2 amarillo
104
105     VERDE1 = 0;
106     AMARILLO1 = 0;
107     ROJO1 = 1;
108
109     VERDE2 = 0;
110     AMARILLO2 = 1;
111     ROJO2 = 0;
112
113     __delay_ms(1000);
114 }
```

```

115     else
116     {
117         // si el interruptor esta apagado
118         // se apagan todos los leds
119         PORTB = 0x00;
120     }
121 }
122
123 return;
124 }

```

6.2 Fase 1

En la primera fase, el semáforo 1 mantiene encendida la luz verde, mientras que el semáforo 2 mantiene encendida la luz roja.

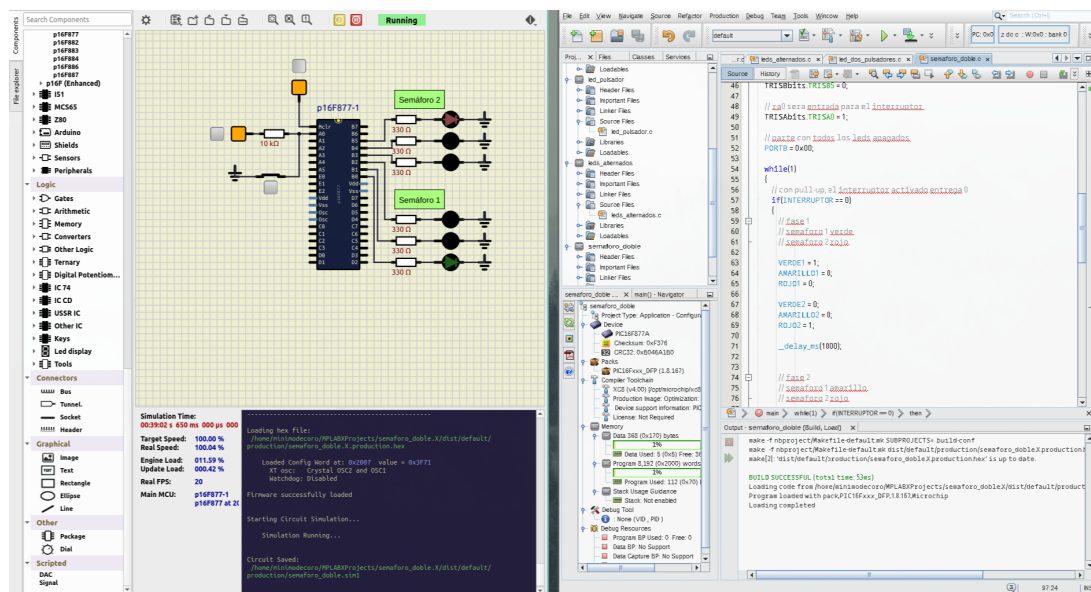


Figura 8: Fase 1: semáforo 1 en verde y semáforo 2 en rojo.

6.3 Fase 2

En la segunda fase, el semáforo 1 cambia a amarillo, mientras que el semáforo 2 continúa en rojo.

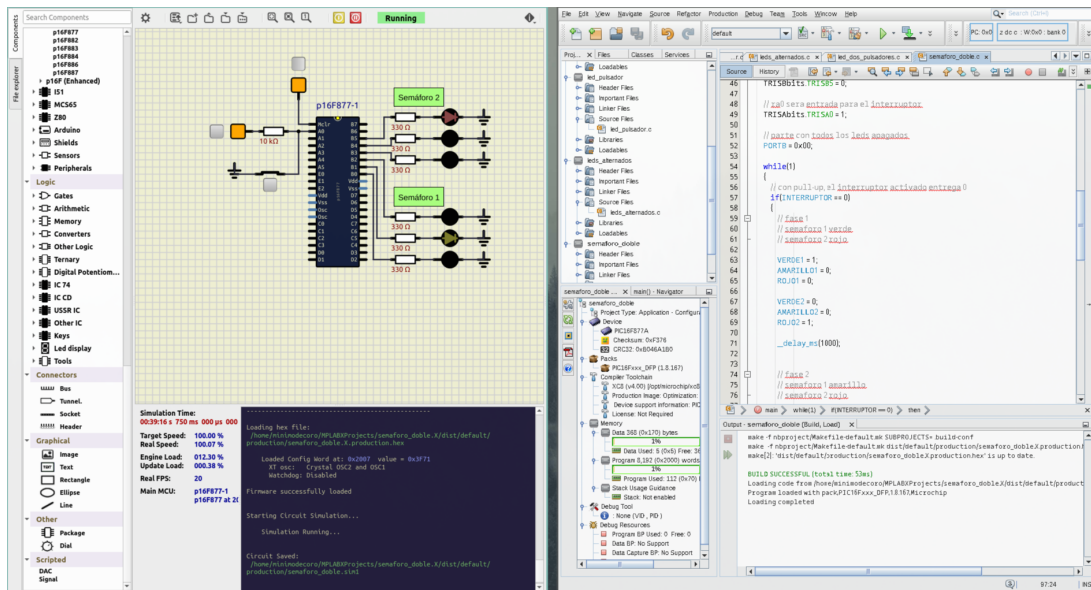


Figura 9: Fase 2: semáforo 1 en amarillo y semáforo 2 en rojo.

6.4 Fase 3

En la tercera fase, el semáforo 1 mantiene encendida la luz roja y el semáforo 2 cambia a verde.

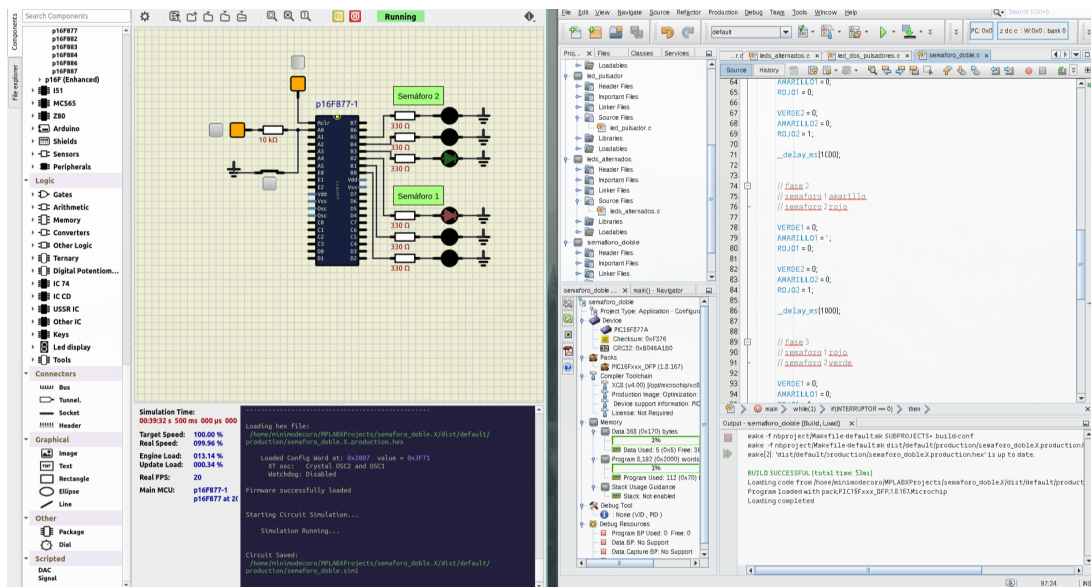


Figura 10: Fase 3: semáforo 1 en rojo y semáforo 2 en verde.

6.5 Fase 4

Finalmente, en la cuarta fase el semáforo 1 permanece en rojo y el semáforo 2 cambia a amarillo.

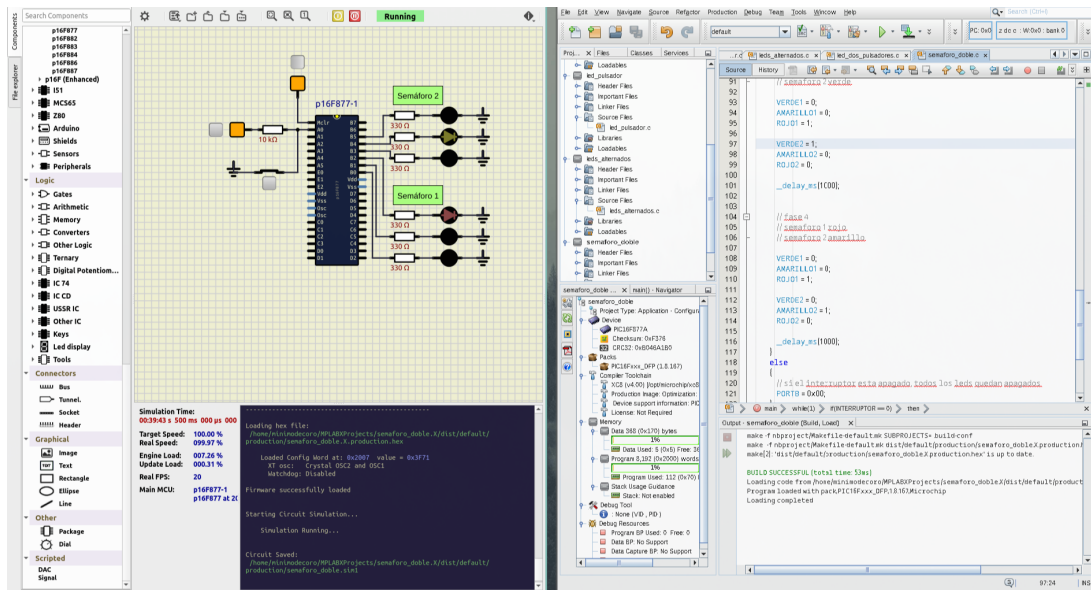


Figura 11: Fase 4: semáforo 1 en rojo y semáforo 2 en amarillo.

7. Dificultades y aprendizajes durante el desarrollo

Durante el desarrollo de las actividades también fueron apareciendo distintas dificultades que nos ayudaron a comprender de mejor manera cómo se relacionan el programa, la configuración del microcontrolador y el circuito simulado.

Una de las primeras confusiones estuvo relacionada con la configuración de los pines como entradas o salidas. Al comenzar las actividades era necesario entender qué significaban los registros TRIS y cómo un valor 0 configura un pin como salida, mientras que un valor 1 lo configura como entrada. Esto se volvió más claro al avanzar desde el primer ejercicio, donde solamente se controlaba un LED, hacia los ejercicios que incorporaban uno o más pulsadores.

Al comenzar a utilizar el puerto A también tuvimos que considerar que algunos de sus pines pueden funcionar como entradas analógicas. Por esta razón fue necesario utilizar la configuración de ADON1 para poder trabajar con RA0 y RA1 como entradas digitales. Este fue un detalle que inicialmente no era tan evidente solamente observando el circuito, ya que también dependía de la configuración realizada desde el programa.

Otra dificultad importante fue comprender la lógica de los pulsadores utilizando resistencias pull-up. En un comienzo podía resultar confuso que un pulsador presionado correspondiera a un estado lógico 0 y que, al estar sin presionar, la entrada tuviera un estado lógico 1. Al probarlo directamente en las simulaciones pudimos relacionar de mejor manera el estado eléctrico de la entrada con las condiciones utilizadas en las estructuras `if` del código.

Durante las simulaciones también aparecieron problemas relacionados con la configuración del entorno. Fue necesario verificar que la frecuencia configurada en SimulIDE coincidiera con los 4 MHz definidos mediante `_XTAL_FREQ`, ya que esta frecuencia se utiliza para calcular correctamente los retardos del programa. También fue necesario mantener el pin MCLR en un nivel lógico alto para evitar que el microcontrolador permaneciera en estado de reset.

Otro aprendizaje fue distinguir entre compilar el proyecto y generar correctamente el archivo `.hex` utilizado por SimulIDE. Para las actividades se utilizó la compilación de producción de MPLAB X y posteriormente se cargó el archivo generado en el microcontrolador de la simulación. Esto permitió comprender un poco mejor el proceso que existe entre escribir el código en C y finalmente ejecutarlo en el PIC.

A medida que aumentó la cantidad de componentes también se volvió más importante mantener un orden entre los pines utilizados en el programa y las conexiones realizadas en el circuito. Por ejemplo, en las actividades con dos pulsadores y posteriormente en el semáforo fue necesario revisar que cada entrada y salida estuviera conectada al pin

definido en el código, ya que una conexión realizada en un pin diferente podía provocar que el programa estuviera correcto pero que la simulación no respondiera como se esperaba.

Finalmente, la actividad de los dos semáforos nos permitió reunir gran parte de lo aprendido anteriormente. Al tener seis LED funcionando como salidas y un interruptor como entrada, fue necesario organizar tanto el circuito como el programa de una forma más clara. La división de la secuencia en cuatro fases permitió entender cómo una serie de instrucciones simples puede utilizarse para construir un comportamiento más completo.

8. Conclusión

En este trabajo pudimos aplicar de forma práctica el manejo de entradas y salidas digitales del PIC16F877A, utilizando MPLAB X junto al compilador XC8 para programar y generar los archivos `.hex`, y SimulIDE para comprobar el funcionamiento de los circuitos.

Primero trabajamos con una salida simple haciendo parpadear un LED, después agregamos pulsadores como entradas y fuimos aumentando progresivamente la cantidad de entradas y salidas utilizadas, hasta llegar al control de dos semáforos.

A través de las actividades pudimos comprender mejor cómo funcionan los registros TRIS para definir si un pin se utiliza como entrada o salida, además del uso de PORT para controlar los estados de los pines. También trabajamos con la configuración de ADCON1 para poder utilizar los pines del puerto A como entradas digitales.

El uso de resistencias pull-up también permitió comprender la lógica utilizada por los pulsadores, donde la entrada permanece normalmente en estado lógico alto y cambia a estado lógico bajo al ser accionada.

Finalmente, la actividad del semáforo permitió integrar los conceptos anteriores en una secuencia de control más completa, utilizando seis salidas, una entrada y tiempos definidos para controlar los diferentes estados de ambos semáforos.

En general, el trabajo nos permitió relacionar de una forma más clara el código realizado en C con el comportamiento de las entradas y salidas del microcontrolador dentro de la simulación. Además, los errores y dificultades que fueron apareciendo durante las actividades fueron útiles para comprender que el funcionamiento de un sistema con microcontroladores no depende solamente del código, sino también de la configuración del PIC, de las conexiones realizadas y de que exista coherencia entre el programa y el circuito.